



杭州电子科技大学

实验七 温度采集控制器实验报告

课程 微机原理与接口技术

学院 自动化学院

专业 自动化（新工科试验班）

学号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 6 月 21 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	总体方案	1
2	硬件电路设计	1
2.1	单片机最小系统	2
2.2	温度采集与 ADC0808 接口	2
2.3	ADC0808 控制时序	3
2.4	数码管显示电路	4
2.5	端口分配	5
3	软件设计	5
3.1	程序设计思路	6
3.2	定时中断流程	7
3.3	温度近似换算	7
3.4	完整程序清单	7
4	仿真调试与结果分析	12
4.1	调试步骤	12
4.2	结果现象	13
4.3	问题分析	13
5	实验总结	13

1 实验要求与实验目的

1.1 实验要求

本实验要求利用 AT89C51 单片机、ADC0808 模数转换器和 AT502 热敏电阻构成温度采集控制器。系统通过热敏电阻分压电路把温度变化转换为模拟电压，再由 ADC0808 转换为 8 位数字量，最后由单片机处理后送至最左侧两位共阴极八段数码管显示。根据实验说明和开发板资源限制，本实验采用如下连接关系：

- 1. ADC0808 的 8 位数据输出口接 AT89C51 的 P1.0–P1.7。
- 2. ADC0808 的 OE 接 P2.1，START 与 ALE 共接 P2.2，EOC 接 P3.4。
- 3. ADC0808 的 CLOCK 引脚由 Proteus 数字时钟源提供 500kHz 方波。
- 4. 温度传感器采用 5kΩ NTC 热敏电阻，经过分压后接 ADC0808 的 IN0 通道。
- 5. 显示部分使用最左边两只共阴极数码管显示温度值。

1.2 实验目的

- 1. 理解 A/D 转换的基本过程，掌握模拟量采样、量化和数字量读取之间的关系。
- 2. 熟悉 ADC0808 的通道选择、启动转换、结束判断和输出允许控制方法。
- 3. 掌握 AT89C51 定时器中断、并行 I/O 口和动态数码管显示的综合应用。
- 4. 理解热敏电阻分压测温的基本原理，并能够在 Proteus 中完成电路与程序联合调试。

1.3 总体方案

系统以 AT89C51 为控制核心。热敏电阻与固定电阻构成分压网络，温度改变时热敏电阻阻值变化，ADC0808 的 IN0 输入电压随之改变。单片机每隔 1s 通过 P2.2 向 ADC0808 发出 START/ALE 脉冲，等待 P3.4 接收到转换完成信号后，打开 P2.1 对应的 OE 控制端，从 P1 口读入 8 位转换结果。程序将 ADC 值近似换算为温度值，并通过 P0 总线和两片 74HC573 锁存器驱动数码管动态显示。

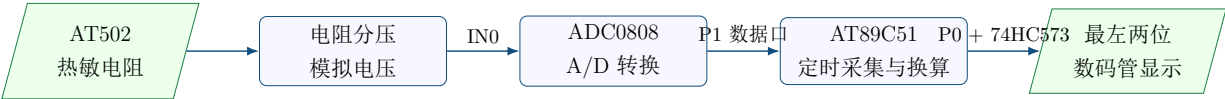
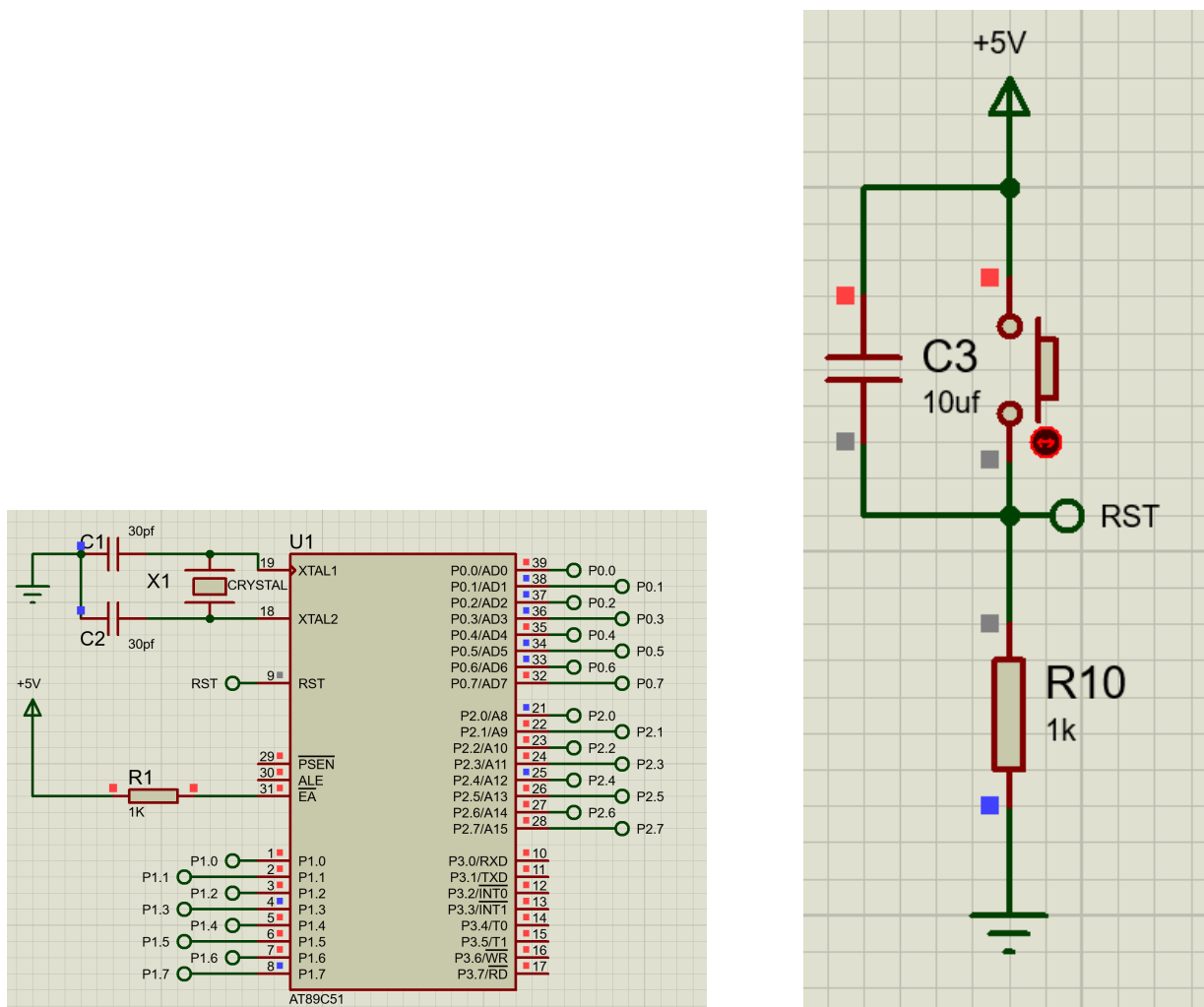


图 1 温度采集控制器总体结构

2 硬件电路设计

2.1 单片机最小系统

AT89C51 采用 +5V 供电，XTAL1 与 XTAL2 外接晶振和两个 30pF 电容构成时钟电路。EA 端通过电阻接 +5V，使单片机从片内程序存储器取指。RST 引脚连接复位网络，保证上电或按键复位时程序能够从 0000H 地址重新执行。该部分是单片机稳定运行的基础。



(a) AT89C51 供电与晶振电路

(b) 复位电路

图 2 单片机最小系统基础电路

2.2 温度采集与 ADC0808 接口

ADC0808 采用 8 位逐次逼近式 A/D 转换结构，具有 IN0-IN7 八路模拟输入。由于本实验只采集一路温度信号，因此 ADD A、ADD B、ADD C 全部接地，固定选择 IN0 通道。VREF(+) 接 +5V，VREF(-) 接 GND，因此转换结果满足近似关系：

$$D = \frac{V_{IN0}}{5V} \times 255$$

其中 D 为 P1 口读到的 8 位 ADC 数值。

温度分压采用固定上拉电阻与 NTC 热敏电阻构成。若上拉电阻为 R_3 ，热敏电阻为 R_T ，则 IN0 电压为：

$$V_{IN0} = 5V \times \frac{R_T}{R_3 + R_T}$$

NTC 热敏电阻温度升高时阻值下降，因此 IN0 电压和 ADC 数值也随温度升高而下降。为了便于仿真调试，可用电位器代替热敏电阻进行阻值变化模拟，但不宜将电位器、固定电阻和热敏电阻随意串联，否则分压关系会偏离测温模型。

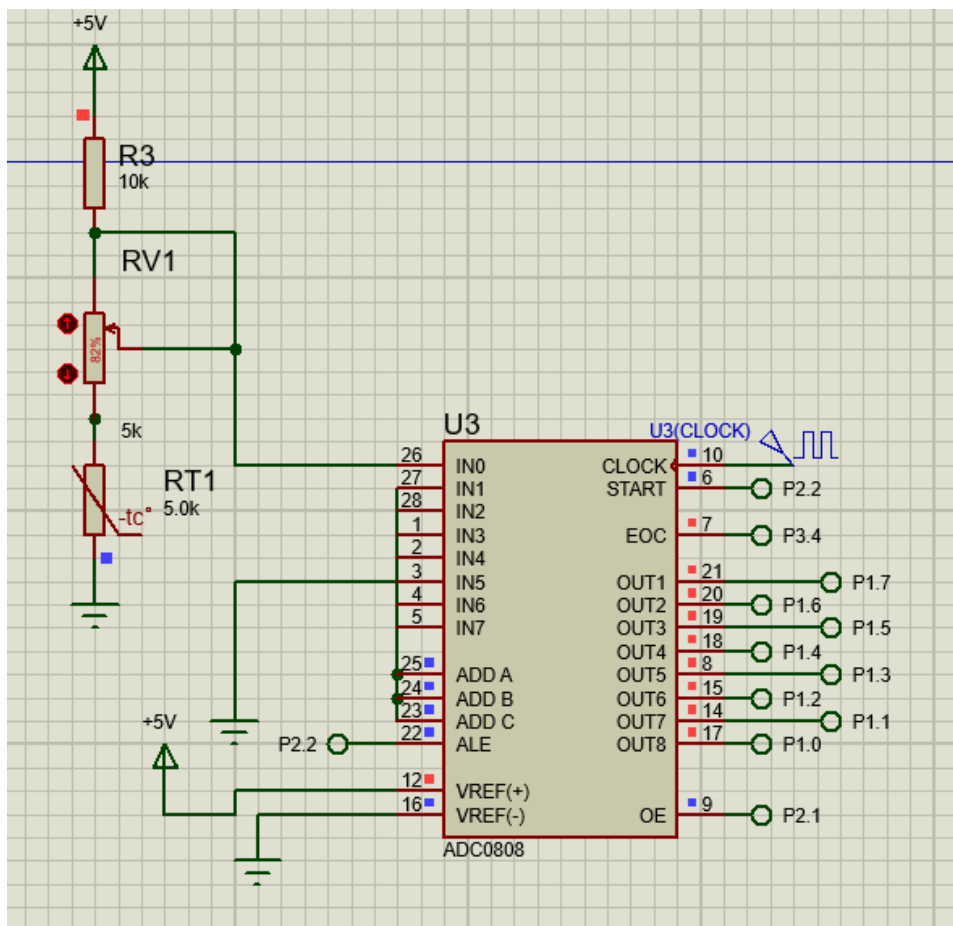


图 3 ADC0808 温度采集接口电路

2.3 ADC0808 控制时序

本实验将 START 与 ALE 共接 P2.2。采样时，单片机先使 P2.2 产生一个高电平脉冲，ADC0808 锁存通道地址并开始转换；随后程序检测 EOC 引脚状态，等待转换完成；最后置位 OE，使 ADC0808 将转换结果输出到 P1 口。读取完成后关闭 OE，避免数据总线长期被外设驱动。

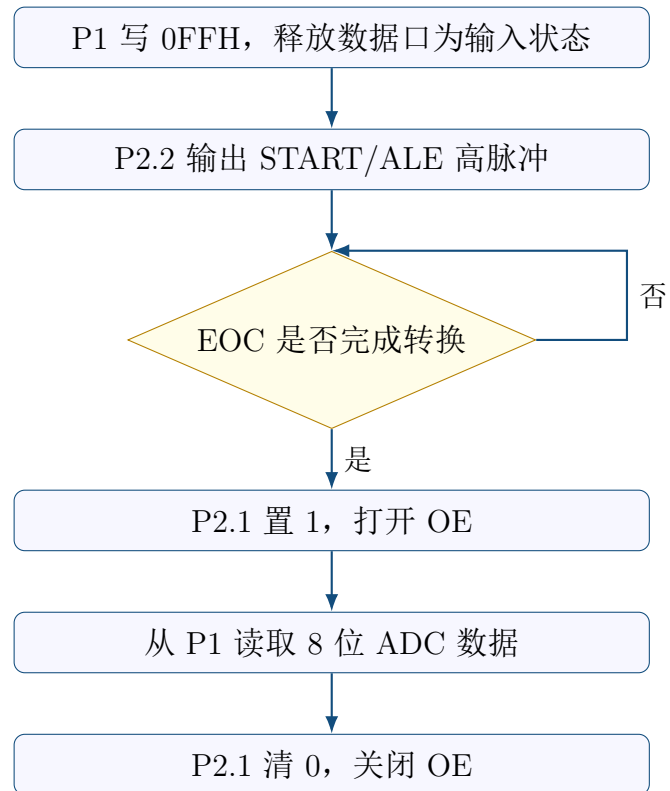


图 4 ADC0808 一次采集流程

2.4 数码管显示电路

显示部分采用一组 8 位共阴极数码管，实验只使用最左边两位。P0 作为共用数据总线，分别送入两片 74HC573：一片锁存段码，另一片锁存位选。P2.6 控制段码锁存，P2.7 控制位选锁存。程序通过快速交替刷新十位和个位，利用视觉暂留形成稳定的两位温度显示。

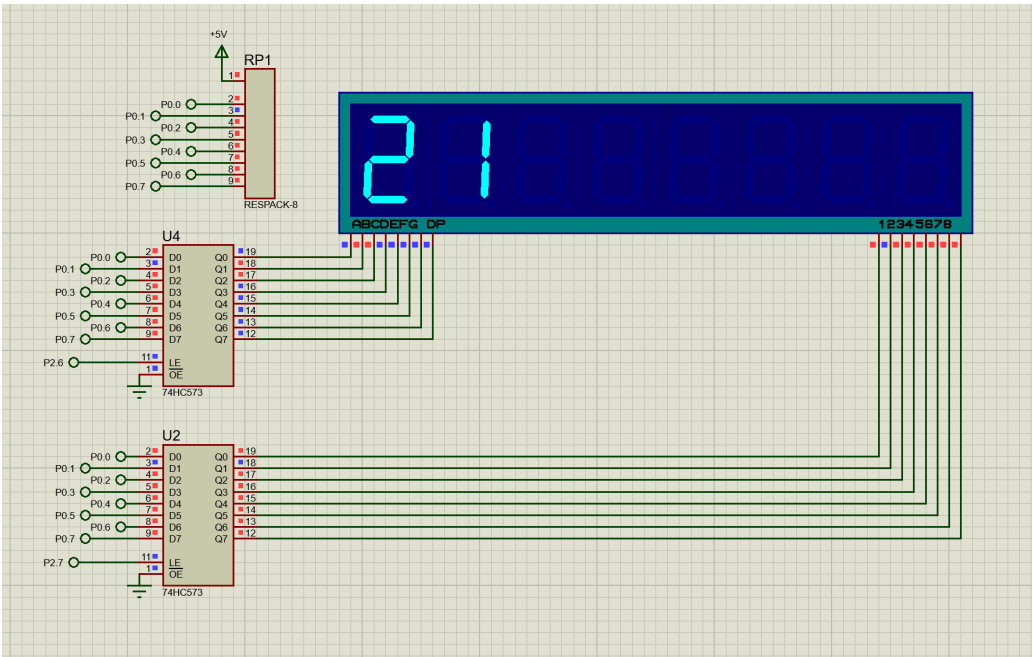


图 5 两位温度显示与 74HC573 锁存电路

2.5 端口分配

表 1 AT89C51 与外设连接关系

单片机端口	外设连接	功能说明
P1.0–P1.7	ADC0808 OUT8–OUT1	读取 8 位 A/D 转换数据，注意 OUT1 为高位
P2.1	ADC0808 OE	输出允许控制，高电平读取数据
P2.2	ADC0808 START、ALE	产生转换启动和地址锁存脉冲
P3.4	ADC0808 EOC	查询 A/D 转换结束状态
P0.0–P0.7	两片 74HC573 输入端	复用输出段码和位选码
P2.6	段码锁存 74HC573 的 LE	锁存数码管段码
P2.7	位选锁存 74HC573 的 LE	锁存数码管位选

3 软件设计

3.1 程序设计思路

程序采用“主循环刷新显示，定时中断周期采集”的结构。主循环不做复杂计算，只不断输出十位和个位的段码、位选码，保证数码管亮度稳定。定时器 0 工作在方式 1，按 50ms 进入一次中断，累计 20 次约为 1s。每满 1s 后，程序启动 ADC0808 转换、读取 P1 数据、近似换算温度，并拆分为十位和个位，供主循环显示。

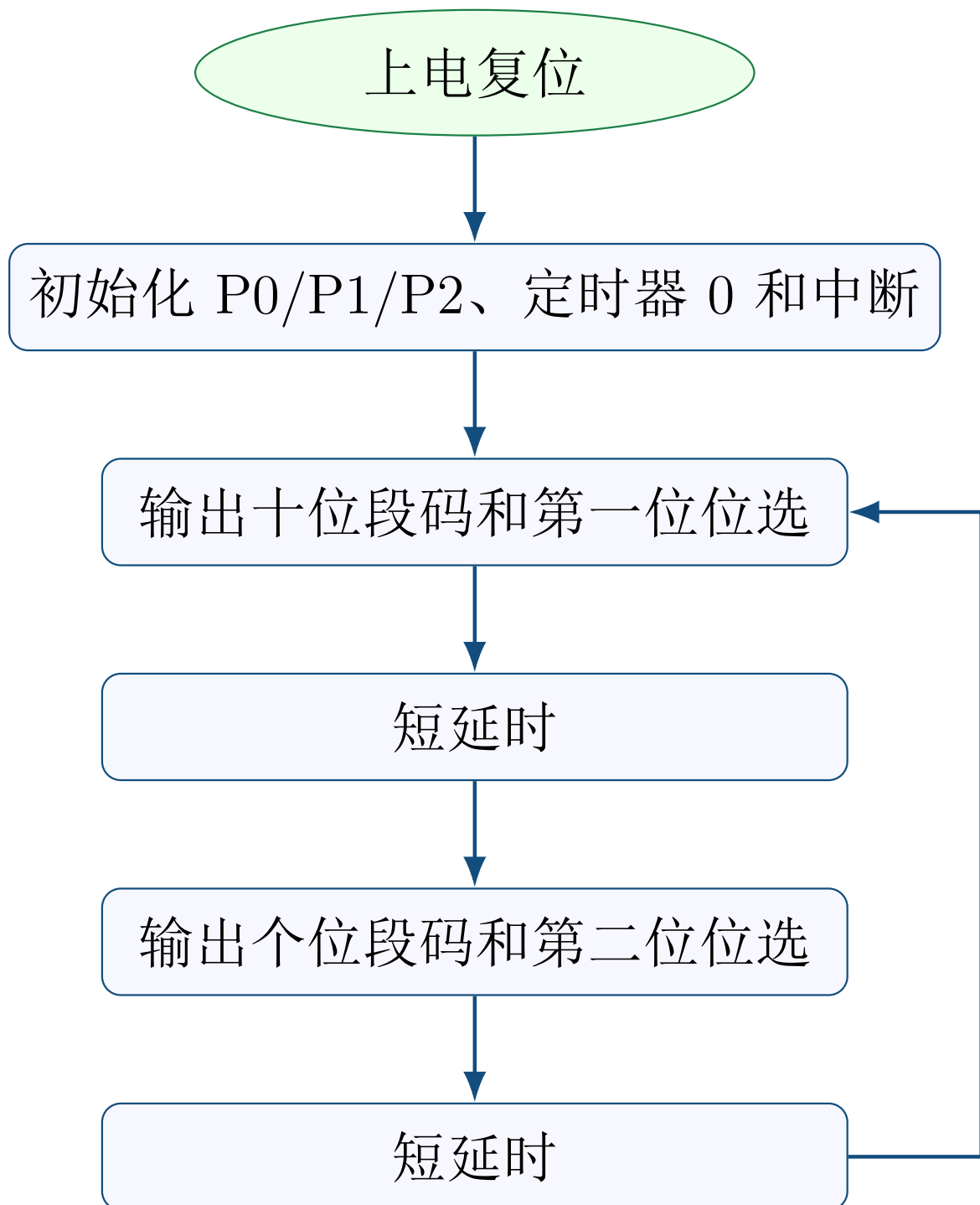


图 6 主程序显示扫描流程图

3.2 定时中断流程

定时中断既负责 1s 采集周期，又避免在主循环中使用长时间等待。中断服务程序重新装载 TH0、TL0 后递减计数变量，若未满 1s 则直接返回；若计数到 0，则重新装载计数值并执行一次 ADC 采集和温度换算。

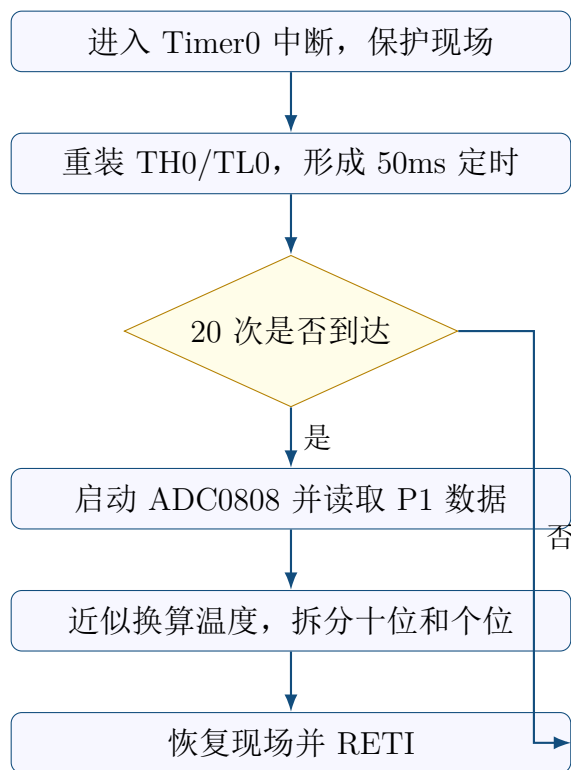


图 7 1s 周期采集的定时中断流程图

3.3 温度近似换算

本实验的重点是掌握 ADC0808 的控制和数据显示，因此程序采用分段近似方式进行温度换算。对于 5kΩ NTC 与 10kΩ 上拉电阻组成的分压电路，25℃ 附近热敏电阻约为 5kΩ，此时 ADC 数值约为：

$$D_{25} = \frac{5k}{10k + 5k} \times 255 \approx 85$$

温度升高时 ADC 值变小，温度降低时 ADC 值变大。程序以 85 为分界点，分别使用线性近似，使显示值能够随采集电压变化而改变。该方法程序短、容易在 ASEM-51 下编译，适合实验验收；若进行实际测温，应进一步使用查表法或 Steinhart-Hart 方程进行标定。

3.4 完整程序清单

Listing 1 07_temperature.A51 完整源程序

```

1 ; =====
2 ; AT89C51 + ADC0808

```

```
3 ; Proteus ASEM-51
4 ;=====
5
6 $NOMOD51
7 $INCLUDE (8051.MCU)
8
9 ADC_OE EQU 0A1H ; P2.1
10 ADC_ST EQU 0A2H ; P2.2 START + ALE
11 ADC_EOC EQU 0B4H ; P3.4
12
13 SEG_LE EQU 0A6H ; P2.6
14 BIT_LE EQU 0A7H ; P2.7
15
16 CNT EQU 30H
17 TEN EQU 31H
18 ONE EQU 32H
19
20 ORG 0000H
21 LJMP START
22
23 ORG 000BH
24 LJMP TO_INT
25
26 ORG 0100H
27
28 START:
29 MOV SP,#60H
30
31 MOV P0,#00H
32 MOV P1,#0FFH
33 MOV P2,#0FFH
34
35 CLR ADC_OE
36 CLR ADC_ST
37
38 MOV CNT,#20
39 MOV TEN,#2
40 MOV ONE,#5
41
42 MOV TMOD,#01H
```

```
43      MOV TH0,#3CH          ; 50 ms, 12 MHz
44      MOV TLO,#0B0H
45
46      SETB ETO
47      SETB EA
48      SETB TRO
49
50 MAIN:
51      ; close all digits
52      MOV PO,#0FFH
53      SETB BIT_LE
54      CLR BIT_LE
55
56      ; show ten
57      MOV DPTR,#TAB
58      MOV A,TEN
59      MOVC A,@A+DPTR
60      MOV PO,A
61      SETB SEG_LE
62      CLR SEG_LE
63
64      MOV PO,#0FEH          ; left first digit
65      SETB BIT_LE
66      CLR BIT_LE
67      LCALL DELAY
68
69      ; close all digits
70      MOV PO,#0FFH
71      SETB BIT_LE
72      CLR BIT_LE
73
74      ; show one
75      MOV DPTR,#TAB
76      MOV A,ONE
77      MOVC A,@A+DPTR
78      MOV PO,A
79      SETB SEG_LE
80      CLR SEG_LE
81
82      MOV PO,#0FDH          ; left second digit
```

```
83      SETB BIT_LE
84      CLR BIT_LE
85      LCALL DELAY
86
87      SJMP MAIN
88
89      ;=====
90      ; Timer0 interrupt: 50 ms once, read ADC every 1 s
91      ;=====
92
93 TO_INT:
94      PUSH ACC
95      PUSH B
96      PUSH PSW
97
98      MOV TH0,#3CH
99      MOV TLO,#0B0H
100
101      DJNZ CNT,TO_EXIT
102      MOV CNT,#20
103
104      ; start ADC0808
105      MOV P1,#0FFH
106      CLR ADC_OE
107
108      SETB ADC_ST
109      NOP
110      NOP
111      CLR ADC_ST
112
113 WAIT0:
114      JB ADC_EOC,WAIT0
115
116 WAIT1:
117      JNB ADC_EOC,WAIT1
118
119      SETB ADC_OE
120      NOP
121      MOV A,P1
122      CLR ADC_OE
```

```
123
124      ; A = ADC value
125      ; if ADC < 85, temperature is higher than 25
126      CJNE A,#85,CHK_ADC
127      SJMP COLD
128
129 CHK_ADC:
130      JC HOT
131
132 COLD:
133      ; temp = (160 - ADC) / 3
134      MOV B,A
135      MOV A,#160
136      CLR C
137      SUBB A,B
138      JNC COLD_OK
139      MOV A,#0
140      SJMP SPLIT
141
142 COLD_OK:
143      MOV B,#3
144      DIV AB
145      SJMP SPLIT
146
147 HOT:
148      ; temp = 25 + (85 - ADC) * 2 / 3
149      MOV B,A
150      MOV A,#85
151      CLR C
152      SUBB A,B
153      MOV B,#2
154      MUL AB
155      MOV B,#3
156      DIV AB
157      ADD A,#25
158
159 SPLIT:
160      ; split temperature to ten and one
161      MOV B,#10
162      DIV AB
```

```
163      MOV TEN,A
164      MOV ONE,B
165
166 TO_EXIT:
167      POP PSW
168      POP B
169      POP ACC
170      RETI
171
172 ;=====
173
174 DELAY:
175      MOV R6,#2
176 D1:
177      MOV R7,#250
178 D2:
179      DJNZ R7,D2
180      DJNZ R6,D1
181      RET
182
183 TAB:
184      DB 3FH,06H,5BH,4FH,66H
185      DB 6DH,7DH,07H,7FH,6FH
186
187      END
```

4 仿真调试与结果分析

4.1 调试步骤

1. 检查 AT89C51 的电源、晶振、复位和 EA 引脚连接，确认单片机可以正常运行程序。
2. 检查 ADC0808 的 VREF(+)、VREF(-)、CLOCK、START/ALE、EOC 和 OE 引脚，确认控制信号符合实验要求。
3. 检查 ADC0808 的 OUT1-OUT8 与 P1 口的连接顺序。ADC0808 的 OUT1 为高位，OUT8 为低位，若位序接反，显示值会出现大幅跳变。
4. 检查 ADD A、ADD B、ADD C 是否接地，确保固定选择 IN0 通道。
5. 加载汇编生成的 HEX 文件，启动仿真，观察最左边两位数码管是否能够显示温度

值。

6. 改变热敏电阻模型或用电位器模拟热敏电阻阻值变化，观察显示温度是否随 ADC 输入变化而更新。

4.2 结果现象

仿真运行后，最左边两位数码管能够稳定显示温度值。程序每 1s 进行一次 ADC 采样，因此改变模拟输入后，显示结果不会立即变化，而是在下一次采样周期到来后更新。显示部分采用动态扫描方式，主循环快速交替刷新十位和个位，因此肉眼观察为两位同时点亮。

4.3 问题分析

调试过程中需要特别注意两个问题。第一，ADC0808 数据位序必须与程序读取方式一致。8051 程序直接执行 MOV A,P1，默认 P1.7 为最高位、P1.0 为最低位；而 ADC0808 的 OUT1 为最高位、OUT8 为最低位，因此硬件连接应按高位对高位、低位对低位的方式处理。第二，用电位器模拟热敏电阻时，应保持分压模型清晰。若将固定电阻、电位器和热敏电阻全部串联，IN0 电压不再对应单一热敏电阻阻值，显示值会出现不易解释的跳变。

表 2 常见调试问题与处理方法

现象	可能原因	处理方法
数码管不亮	位选方向错误或锁存信号未连接	调整位选码，检查 P2.6/P2.7
显示值跳变很大	ADC0808 输出位序接反	将 OUT1 接 P1.7, OUT8 接 P1.0
显示一直为 00	IN0 电压超出程序换算范围或 OE 未打开	检查分压电阻和 P2.1 控制
调节电位器非线性	电位器与热敏电阻串联方式不当	用电位器单独模拟热敏电阻
采样无变化	EOC 等待逻辑或 START 脉冲异常	检查 P2.2、P3.4 和 500kHz 时钟

5 实验总结

本实验完成了基于 AT89C51 和 ADC0808 的温度采集显示系统设计。硬件方面，系统包含单片机最小系统、NTC 热敏电阻分压电路、ADC0808 转换接口和两位数码管显示电路；软件方面，程序利用定时器中断实现 1s 采集周期，并在主循环中完成数码管动态扫

描。通过本实验，进一步理解了 ADC0808 的启动、转换完成判断和数据读取过程，也认识到模拟量采集电路中分压电阻、位序连接和参考电压对结果准确性的影响。

实验中采用简化的分段线性换算实现温度显示，能够满足课堂验收中“采集、转换、显示”的功能要求。若将该系统用于更高精度的实际温度测量，还应根据热敏电阻的具体参数进行标定，并使用查表或非线性公式提高温度换算精度。